

Plug

插头 DP

定义

有些 [状压 DP](#) 问题要求我们记录状态的连通性信息，这类问题一般被形象的称为插头 DP 或连通性状态压缩 DP。例如格点图的哈密顿路径计数，求棋盘的黑白染色方案满足相同颜色之间形成一个连通块的方案数，以及特定图的生成树计数等等。这些问题通常需要我们对状态的连通性进行编码，讨论状态转移过程中连通性的变化。

引入

骨牌覆盖与轮廓线 DP

温故而知新，在开始学习插头 DP 之前，不妨先让我们回顾一个经典问题。

▼ 例题 [「HDU 1400」Mondriaan's Dream](#)

题目大意：在 的棋盘内铺满 或 的多米诺骨牌，求方案数。

当 或 规模不大的时候，这类问题可以使用 [状压 DP](#) 解决。逐行划分阶段，设 表示当前已考虑过前 行，且第 行的状态为 的方案数。这里的状态 的每一位可以表示这个这个位置是否已被上一行覆盖。

	1	2	3	4	5	6
1	1	1	0	0	1	1
2	0	0	1	1	1	1
3	1	1	0	0	0	0
4	1	1	1	1	1	1
5	1	1	0	0	1	1
6	1	1	1	1	1	1

另一种划分阶段的方法是逐格 DP，或者称之为轮廓线 DP。表示已经考虑到第 行第 列，且当前轮廓线上的状态为 的方案数。

虽然逐格 DP 中我们的状态增加了一个维度，但是转移的时间复杂度减少为 ，所以时间复杂度未变。我们用 表示当前阶段的状态，用 表示下一阶段的状态， 表示当前枚举的函数值，那么有如下的状态转移方程：

```
1 if (s >> j & 1) {           // 如果已被覆盖
2   f1[s ^ 1 << j] += u;      // 不放
3 } else {                     // 如果未被覆盖
4   if (j != m - 1 && (!(s >> j + 1 & 1))) f1[s ^ 1 << j + 1] += u; // 横放
5   f1[s ^ 1 << j] += u;      // 竖放
6 }
```

观察到这里不放和竖放的方程可以合并。

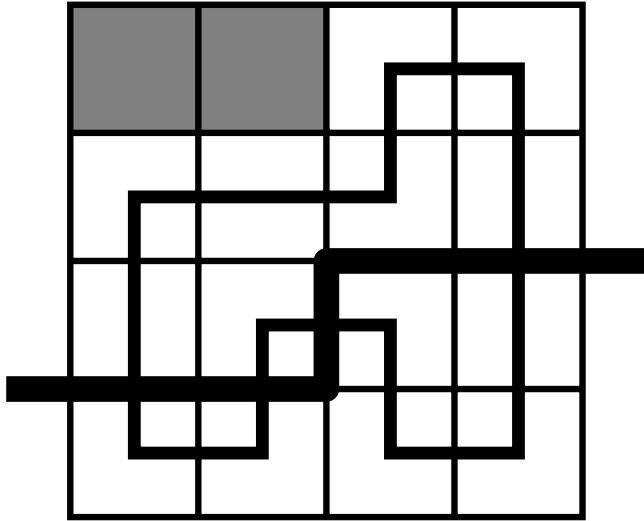
► 实现

► 习题 [「SRM 671. Div 1 900」BearDestroys](#)

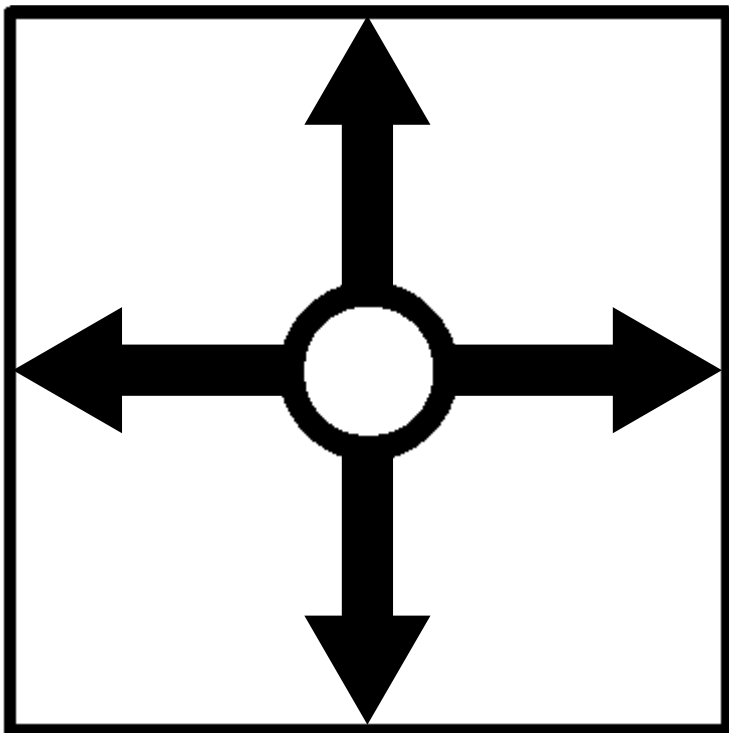
术语

阶段：动态规划执行的顺序，后续阶段的结果只与前序阶段的结果有关（无后效性）。很多 DP 问题可以有多种划分阶段的方式。例如在背包问题中，我们通常既可以按照物品划分阶段，也可以按照背包容量划分阶段（外层循环先枚举什么）。而在多米诺骨牌问题中，我们可以按照行、列、格子以及对角线等特征划分阶段。

轮廓线：已决策状态和未决策状态的分界线。



插头：一个格子某个方向的插头存在，表示这个格子在这个方向与相邻格子相连。



路径模型

多条回路

例题

▼ 例题 「HDU 1693」 Eat the Trees

题目大意：求用若干条回路覆盖 棋盘的方案数，有些位置有障碍。

严格来说，多条回路问题并不属于插头 DP，因为我们只需要和上面的骨牌覆盖问题一样，记录插头是否存在，然后成对的合并和生成插头就可以了。

注意对于一个宽度为 的棋盘，轮廓线的宽度为 ，因为包含 个上插头，和 个左插头。注意，当一行迭代完成之后，最右边的左插头通常是不合法的状态，同时我们需要补上下一行第一个左插头，这需要我们调整当前轮廓线的状态，通常是所有状态进行左移，我们把这个操作称为滚动 `roll()`。

► 例题代码

习题

► 习题 [「ZJU 3466」The Hive II](#)

一条回路

例题

▼ 例题 [「Andrew Stankevich Contest 16 - Problem F」Pipe Layout](#)

题目大意：求用一条回路覆盖 棋盘的方案数。

在上面的状态表示中我们每合并一组连通的插头，就会生成一条独立的回路，因而在本题中，我们还需要区分插头之间的连通性（出现了！）。这我们需要对状态进行额外的编码。

状态编码

通常的编码方案有括号表示和最小表示，这里着重介绍泛用性更好的最小表示。我们用长度 的整形数组，记录轮廓线上每个插头的状态， 表示没有插头，并约定连通的插头用相同的数字进行标记。

那么下面两组编码方式表示的是相同的状态：

- 0 3 1 0 1 3
- 0 1 2 0 2 1

我们将相同的状态都映射成字典序最小表示，例如在上例中的 0 1 2 0 2 1 就是一组最小表示。

我们用 `b[]` 数组表示轮廓线上插头的状态。`bb[]` 表示在最小表示的编码的过程中，每个数字被映射到的最小数字。注意 表示插头不存在，不能被映射成其他值。

► 代码实现

我们注意到插头总是成对出现，成对消失的。因而 0 1 2 0 1 2 这样的状态是不合法的。合法的状态构成一组括号序列，实际中合法状态可能是非常稀疏的。

手写哈希

在一些 [状压 DP](#) 的问题中，合法的状态可能是稀疏的（例如本题），为了优化时空复杂度，我们可以使用哈希表存储合法的 DP 状态。对于 C++ 选手，我们可以使用 [std::unordered_map](#)，当然也可以直接手写，这样可以灵活的将状态转移函数也封装于其中。

▼ 代码实现

```

1 constexpr int MaxSZ = 16796, Prime = 9973;
2
3 struct hashTable {
4     int head[Prime], next[MaxSZ], sz;
5     int state[MaxSZ];
6     long long key[MaxSZ];
7
8     void clear() {
9         sz = 0;
10        memset(head, -1, sizeof(head));
11    }
12
13    void push(int s) {
14        int x = s % Prime;
15        for (int i = head[x]; ~i; i = next[i]) {
16            if (state[i] == s) {
17                key[i] += d;
18                return;
19            }
20        }
21        state[sz] = s, key[sz] = d;
22        next[sz] = head[x];
23        head[x] = sz++;
24    }
25
26    void roll() { REP(i, sz) state[i] <=< offset; }
27 } H[2], *H0, *H1;

```

上面的代码中：

- `MaxSZ` 表示合法状态的上界，可以估计，也可以预处理出较为精确的值。
- `Prime` 一个小于 `MaxSZ` 的大素数。
- `head[]` 表头节点的指针。
- `next[]` 后续状态的指针。
- `state[]` 节点的状态。
- `key[]` 节点的关键字，在本题中是方案数。
- `clear()` 初始化函数，和手写邻接表类似，我们只需要初始化表头节点的指针。
- `push()` 状态转移函数，其中 `d` 是一个全局变量（偷懒），表示每次状态转移所带来的增量。如果找到的话就 `+=`，否则就创建一个状态为 `s`，关键字为 `d` 的新节点。
- `roll()` 迭代完一整行之后，滚动轮廓线。

关于哈希表的复杂度分析，以及开哈希和闭哈希的不同，可以参见 [《算法导论》](#) 中关于散列表的相关章节。

状态转移

▼ 代码实现

```

1 REP(ii, H0->sz) {
2     decode(H0->state[ii]);           // 取出状态，并解码
3     d = H0->key[ii];                 // 得到增量 delta
4     int lt = b[j], up = b[j + 1];    // 左插头，上插头
5     bool dn = i != n - 1, rt = j != m - 1; // 下插头，右插头
6     if (lt && up) {                  // 如果左、上均有插头
7         if (lt == up) {              // 来自同一个连通块
8             if (i == n - 1 &&
9                 j == m - 1) { // 只有在最后一个格子时，才能合并，封闭回路。
10                 push(j, 0, 0);
11             }
12         } else { // 否则，必须合并这两个连通块，因为本题中需要回路覆盖
13             REP(i, m + 1) if (b[i] == lt) b[i] = up;
14             push(j, 0, 0);
15         }
16     } else if (lt || up) { // 如果左、上之中有一个插头
17         int t = lt | up;    // 得到这个插头
18         if (dn) {           // 如果可以向下延伸
19             push(j, t, 0);
20         }
21         if (rt) { // 如果可以向右延伸
22             push(j, 0, t);
23         }
24     } else { // 如果左、上均没有插头
25         if (dn && rt) { // 生成一对新插头
26             push(j, m, m);
27         }
28     }
29 }

```

► 例题代码

习题

- 习题 [「Ural 1519」Formula 1](#)
- 习题 [「USACO 5.4.4」Betsy's Tours](#)
- 习题 [「POJ 1739」Tony's Tour](#)
- 习题 [「USACO 6.1.1」Postal Vans](#)
- 习题 [「HNOI 2007」神奇游乐园](#)
- 习题 [「ProjectEuler 393」Migrating ants](#)

一条路径

例题

▼ 例题 [「ZOJ 3213」Beautiful Meadow](#)

题目大意：一个 $n \times m$ 的方阵 $(n, m \leq 100)$ ，每个格点有一个权值，求一段路径，最大化路径覆盖的格点的权值和。

本题是标准的一条路径问题，在一条路径问题中，编码的状态中还会存在不能配对的独立插头。需要在状态转移函数中，额外讨论独立插头的生成、合并与消失的情况。独立插头的生成和消失对应着路径的一端，因而这类事件不会发生超过两次（一次生成一次消失，或者两次生成一次合并），否则最终结果一定会出现多个连通块。

我们需要在状态中额外记录这类事件发生的总次数，可以将这个信息编码进状态里（注意，类似这样的额外信息在调整轮廓线的时候，不需要跟着滚动），当然也可以在 `hashTable` 数组的外面加维。下面的范例程序中我们选择后者。

状态转移

▼ 代码实现

```

1 REP(i, n) {
2   REP(j, m) {
3     checkMax(ans, A[i][j]); // 需要单独处理一个格子的情况
4     if (!A[i][j]) continue; // 如果有障碍，则跳过，注意这时状态数组不需要滚动
5     swap(H0, H1);
6     REP(c, 3)
7       H1[c].clear(); // c 表示生成和消失事件发生的总次数，最多不超过 2 次
8     REP(c, 3) REP(ii, H0[c].sz) {
9       decode(H0[c].state[ii]);
10      d = H0[c].key[ii] + A[i][j];
11      int lt = b[j], up = b[j + 1];
12      bool dn = A[i + 1][j], rt = A[i][j + 1];
13      if (lt && up) {
14        if (lt == up) { // 在一条路径问题中，我们不能合并相同的插头。
15          // Cannot deploy here...
16        } else { // 有可能参与合并的两者中有独立插头，但是也可以用同样的代码片段处理
17          REP(i, m + 1) if (b[i] == lt) b[i] = up;
18          push(c, j, 0, 0);
19        }
20      } else if (lt || up) {
21        int t = lt | up;
22        if (dn) {
23          push(c, j, t, 0);
24        }
25        if (rt) {
26          push(c, j, 0, t);
27        }
28        // 一个插头消失的情况，如果是独立插头则意味着消失，如果是成对出现的插头则相当于生成了一个独立插头，
29        // 无论哪一类事件都需要将 c + 1。
30        if (c < 2) {
31          push(c + 1, j, 0, 0);
32        }
33      } else {
34        d -= A[i][j];
35        H1[c].push(H0[c].state[ii]);
36        d += A[i][j]; // 跳过插头生成，本题中不要求全部覆盖
37        if (dn && rt) { // 生成一对插头
38          push(c, j, m, m);
39        }
40        if (c < 2) { // 生成一个独立插头
41          if (dn) {
42            push(c + 1, j, m, 0);
43          }
44          if (rt) {
45            push(c + 1, j, 0, m);
46          }
47        }
48      }
49    }
50  }
51  REP(c, 3) H1[c].roll(); // 一行结束，调整轮廓线
52 }

```

► 例题代码

习题

► 习题 [「BZOJ 2310」ParkII](#)

► 习题 [「NOI 2010 Day2」旅行路线](#)

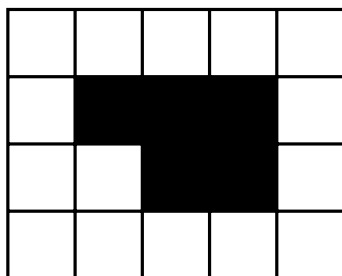
染色模型

除了路径模型之外，还有一类常见的模型，需要我们对棋盘进行染色，相邻的相同颜色节点被视为连通。在路径类问题中，状态转移的时候我们枚举当前路径的方向，而在染色类问题中，我们枚举当前节点染何种颜色。在染色模型中，状态中处在相同连通性的节点可能不止两个。但总体来说依然大同小异。我们不妨来看一个经典的例题。

例题「UVa 10572」Black & White

▼ 例题「UVa 10572」Black & White

题目大意：在 的棋盘内对未染色的格点进行黑白染色，要求所有黑色区域和白色区域连通，且任意一个 的子矩形内的颜色不能完全相同（例如下图中的情况非法），求合法的方案数，并构造一组合法的方案。



状态编码

我们先考虑状态编码。不考虑连通性，那么就是 [SGU 197. Nice Patterns Strike Back](#)，不难用 [状压 DP](#) 直接解决。现在我们需要在状态中同时体现颜色和连通性的信息，考察轮廓线上每个位置的状态，二进制的每 `Offset` 位描述轮廓线上的一个位置，因为只有黑白两种颜色，我们用最低位的奇偶性表示颜色，其余部分示连通性。

考虑第一行上面的节点，和第一列左侧节点，如果要避免特判的话，可以考虑引入第三种颜色区分它们，这里我们观察到这些边界状态的连通性信息一定为 0，所以不需要对第三种颜色再进行额外编码。

在路径问题中我们的轮廓线是由 个上插头与 个左插头组成的。本题中，由于我们还需要判断当前格点为右下角的 子矩形是否合法，所以需要记录左上角格子的颜色，因此轮廓线的长度依然是 。

这样的编码方案中依然保留了很多冗余信息，（连通的区域颜色一定相同，且左上角的格子只需要颜色信息不需要连通性），但是因为已经用了哈希表和最小表示，对时间复杂度的影响不大，为了降低编程压力，就不再细化了。

在最多情况下（例如第一行黑白相间），每个插头的连通性信息都不一样，因此我们需要 位二进制位记录连通性，再加上颜色信息，本题的 `Offset` 为 位。

▼ 代码实现

```

1 constexpr int Offset = 5, Mask = (1 << Offset) - 1;
2 int c[N + 2];
3 int b[N + 2], bb[N + 3];
4
5 T_state encode() {
6     T_state s = 0;
7     memset(bb, -1, sizeof(bb));
8     int bn = 1;
9     bb[0] = 0;
10    for (int i = m; i >= 0; --i) {
11        #define bi bb[b[i]]
12        if (!~bi) bi = bn++;
13        s <<= Offset;
14        s |= (bi << 1) | c[i];
15    }
16    return s;
17 }
18
19 void decode(T_state s) {
20     REP(i, m + 1) {
21         b[i] = s & Mask;
22         c[i] = b[i] & 1;
23         b[i] >>= 1;
24         s >>= Offset;
25     }
26 }

```

手写哈希

因为需要构造任意一组方案，这里的哈希表我们需要添加一组域 `pre[]` 来记录每个状态在上一阶段的任意一个前驱。

▼ 代码实现


```

1 constexpr int Prime = 9979, MaxSZ = 1 << 20;
2
3 template <class T_state, class T_key>
4 struct hashTable {
5     int head[Prime];
6     int next[MaxSZ], sz;
7     T_state state[MaxSZ];
8     T_key key[MaxSZ];
9     int pre[MaxSZ];
10
11 void clear() {
12     sz = 0;
13     memset(head, -1, sizeof(head));
14 }
15
16 void push(T_state s, T_key d, T_state u) {
17     int x = s % Prime;
18     for (int i = head[x]; ~i; i = next[i]) {
19         if (state[i] == s) {
20             key[i] += d;
21             return;
22         }
23     }
24     state[sz] = s, key[sz] = d, pre[sz] = u;
25     next[sz] = head[x], head[x] = sz++;
26 }
27
28 void roll() { REP(ii, sz) state[ii] <= Offset; }
29 };
30
31 hashTable<T_state, T_key> _H, H[N][N], *H0, *H1;

```

方案构造

有了上面的信息，我们就可以容易的构造方案了。首先遍历当前哈希表中的状态，如果连通块数目不超过 $\frac{N}{2}$ ，那么统计进方案数。如果方案数不为 0，我们倒序用 `pre` 数组构造出方案，注意每一行的末尾因为我们执行了 `Roll()` 操作，颜色需要取 `c[j+1]`。

▼ 代码实现

```

1 void print() {
2     T_key z = 0;
3     int u;
4     REP(i, H1->sz) {
5         decode(H1->state[i]);
6         if (*max_element(b + 1, b + m + 1) <= 2) {
7             z += H1->key[i];
8             u = i;
9         }
10    }
11    cout << z << endl;
12    if (z) {
13        DWN(i, n, 0) {
14            B[i][m] = 0;
15            DWN(j, m, 0) {
16                decode(H[i][j].state[u]);
17                int cc = j == m - 1 ? c[j + 1] : c[j];
18                B[i][j] = cc ? 'o' : '#';
19                u = H[i][j].pre[u];
20            }
21        }
22        REP(i, n) puts(B[i]);
23    }
24    puts("");
25 }

```

状态转移

我们记：

- cc 当前正在染色的格子的颜色
- lf 左边格子的颜色
- up 上边格子的颜色
- lu 左上格子的颜色

我们用 表示颜色不存在。接下来讨论状态转移，一共有三种情况，合并，继承与生成：

▼ 状态转移 - 代码

```

1 void trans(int i, int j, int u, int cc) {
2     decode(H0->state[u]);
3     int lf = j ? c[j - 1] : -1, lu = b[j] ? c[j] : -1,
4         up = b[j + 1] ? c[j + 1] : -1; // 没有颜色也是颜色的一种!
5     if (lf == cc && up == cc) { // 合并
6         if (lu == cc) return; // 2x2 子矩形相同的情况
7         int lf_b = b[j - 1], up_b = b[j + 1];
8         REP(i, m + 1) if (b[i] == up_b) { b[i] = lf_b; }
9         b[j] = lf_b;
10    } else if (lf == cc || up == cc) { // 继承
11        if (lf == cc)
12            b[j] = b[j - 1];
13        else
14            b[j] = b[j + 1];
15    } else { // 生成
16        if (i == n - 1 && j == m - 1 && lu == cc) return; // 特判
17        b[j] = m + 2;
18    }
19    c[j] = cc;
20    if (!ok(i, j, cc)) return; // 判断是否会因生成封闭的连通块导致不合法
21    H1->push(encode(), H0->key[u], u);
22 }

```

对于最后一种情况需要注意的是，如果已经生成了一个封闭的连通区域，那么我们不能再用她的颜色染色，否则这种颜色会出现两个连通块。我们似乎需要记录这种事件，可以参考 [「ZOJ 3213」Beautiful Meadow](#) 中的做法，再开一维记录这个事件。不过利用本题的特殊性，我们也可以特判掉。

▼ 特判 - 代码

```
1 bool ok(int i, int j, int cc) {
2     if (cc == c[j + 1]) return true;
3     int up = b[j + 1];
4     if (!up) return true;
5     int c1 = 0, c2 = 0;
6     REP(i, m + 1) if (i != j + 1) {
7         if (b[i] == b[j + 1]) { // 连通性相同，颜色一定相同
8             assert(c[i] == c[j + 1]);
9         }
10        if (c[i] == c[j + 1] && b[i] == b[j + 1]) ++c1;
11        if (c[i] == c[j + 1]) ++c2;
12    }
13    if (!c1) { // 如果会生成新的封闭连通块
14        if (c2) return false; // 如果轮廓线上还有相同的颜色
15        if (i < n - 1 || j < m - 2) return false;
16    }
17    return true;
18 }
```

进一步讨论连通块消失的情况。每当我们对一个格子进行染色后，如果没有其他格子与其上侧的格子连通，那么会形成一个封闭的连通块。这个事件仅在最后一行的最后两列时可以发生，否则后续为了不出现 的同色连通块，这个颜色一定会再次出现，除了下面的情况：

```
1 2 2
2 o#
3 #o
```

我们特判掉这种情况，这样在本题中，就可以偷懒不用记录之前是否已经生成了封闭的连通块了。

► 例题代码

习题

- 习题 [「Topcoder SRM 312. Div1 Hard」CheapestIsland](#)
- 习题 [「JLOI 2009」神秘的生物](#)
- 习题 [「AtCoder Beginner Contest 211. Problem E」Red Polyomino](#)

图论模型

▼ 例题 [「NOI 2007 Day2」生成树计数](#)

题目大意：某类特殊图的生成树计数，每个节点恰好与其前 个节点之间有边相连。

▼ 例题 [「2015 ACM-ICPC Asia Shenyang Regional Contest - Problem E」Efficient Tree](#)

题目大意：给出一个 的网格图，以及相邻四连通格子之间的边权。对于一颗生成树，每个节点的得分为 $1 + [\text{有一条连向上的边}] + [\text{有一条连向左的边}]$ 。生成树的得分为所有节点的得分之积。

你需要求出：最小生成树的边权和，以及所有最小生成树的得分之和。（）

实战篇

例题

▼ 例题 [「HDU 4113」Construct the Great Wall](#)

题目大意：在 的棋盘内构造一组回路，分割所有的 $\times$ 和 $\circ$ 。

有一类插头 DP 问题要求我们在棋盘上构造一组墙，以分割棋盘上的某些元素。不妨称之为修墙问题，这类问题既可视作染色模型，也可视作路径模型。

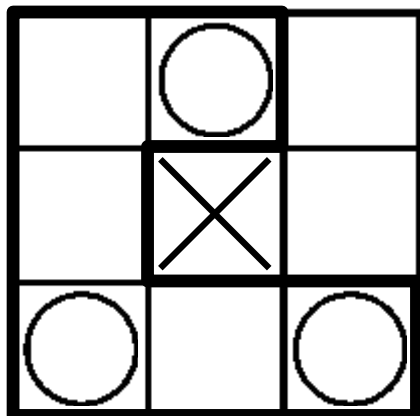


Figure.1

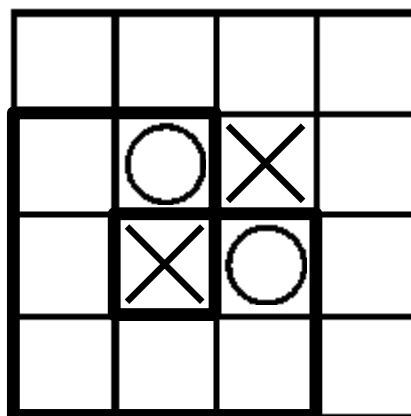


Figure.2

在本题中，如果视作染色模型的话，不仅需要额外讨论染色区域的周长，还要判断在角上触碰而导致不合法的情况（图 2）。另外与 [「UVa 10572」Black & White](#) 不同的是，本题中要求围墙为简单多边形，因而对于下面的回字形的情况，在本题中是不合法的。

```
1 3 3
2 ooo
3 oxo
4 ooo
```

因而我们使用路径模型，转化为 [一条回路](#) 来处理。

我们沿着棋盘的交叉点进行 DP（因而长宽需要增加），每次转移时，需要保证所有的 $\times$ 在回路之外， $\circ$ 在回路之内。因此我们还需要维护当前位置是否在回路内部。对于这个信息我们可以加维，也可以直接统计轮廓线上到这个位置之前出现下插头次数的奇偶性（射线法）。

► 例题代码

习题

- 习题 [「SCOI 2011」地板](#)
- 习题 [「HDU 4796」Winter's Coming](#)
- 习题 [「ZOJ 2125」Rocket Mania](#)
- 习题 [「ZOJ 2126」Rocket Mania Plus](#)
- 习题 [「World Finals 2009/2010 Harbin」Channel](#)
- 习题 [「HDU 3958」Tower Defence](#)
- 习题 [「UVa 10531」Maze Statistics](#)
- 习题 [「Aizu 2452」Pipeline Plans](#)
- 习题 [「SDOI 2014」电路板](#)
- 习题 [「SPOJ CAKE3」Delicious Cake](#)

本章注记

插头 DP 问题通常编码难度较大，讨论复杂，因而属于 OI/ACM 中相对较为 [偏门的领域](#)。这方面最为经典的资料，当属 2008 年 [陈丹琦](#) 的集训队论文——[基于连通性状态压缩的动态规划问题](#)。其次，HDU 的 notonlysuccess 2011 年曾经在博客中连续写过两篇由浅入深的专题，也是不可多得的好资料，不过现在需要在 Web Archive 里考古。

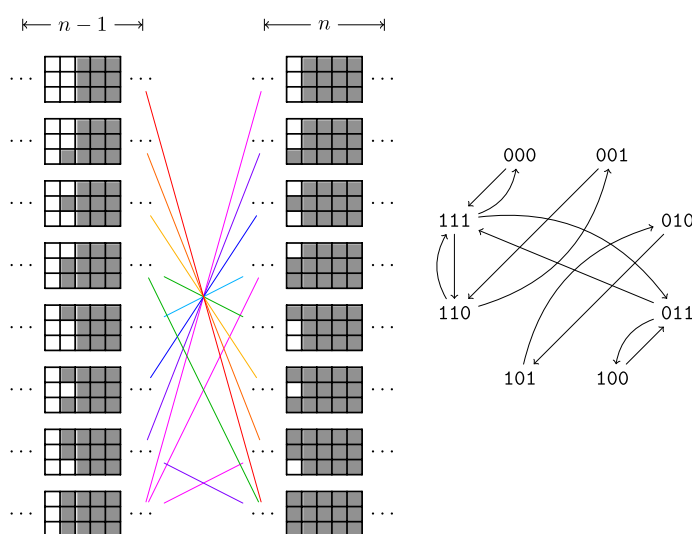
- [notonlysuccess, 【专辑】插头 DP](#)
- [notonlysuccess, 【完全版】插头 DP](#)

多米诺骨牌覆盖

「[HDU 1400](#)」[Mondriaan's Dream](#) 也出现在 [《算法竞赛入门经典训练指南》](#) 中，并作为《轮廓线上的动态规划》一节的例题。[多米诺骨牌覆盖 \(Domino tiling\)](#) 是一组非常经典的数学问题，稍微修改其数据范围就可以得到不同难度，需要应用不同的算法解决的子问题。

当限定 n 时，多米诺骨牌覆盖等价于斐波那契数列。[《具体数学》](#) 中使用了该问题以引出斐波那契数列，并使用了多种方法得到其解析解。

当 n 较大时，可以将转移方程预处理成矩阵形式，并使用 [矩阵乘法进行加速](#)。



当 n 较大时，可以用 [FKT Algorithm](#) 计算其所对应平面图的完美匹配数。

- [「51nod 1031」骨牌覆盖](#)
- [「51nod 1033」骨牌覆盖 V2](#) | [「Vijos 1194」Domino](#)
- [「51nod 1034」骨牌覆盖 V3](#) | [「Ural 1594」Aztec Treasure](#)
- [Wolfram MathWorld, Chebyshev Polynomial of the Second Kind](#)

一条路径

「一条路径」是 [哈密顿路径 \(Hamiltonian Path\)](#) 问题在 [格点图 \(Grid Graph\)](#) 中的一种特殊情况。哈密顿路径的判定性问题是 [NP-complete](#) 家族中的重要成员。

本页面最近更新：2024/11/19 22:34:33, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[Alphnia](#), [billchenchina](#), [CCXXXI](#), [Chrogeek](#), [diauweb](#), [Early0v0](#), [Enter-tainer](#), [gi-b716](#), [iamtwz](#), [ImpleLee](#), [lr1d](#), [isdanni](#), [kenlig](#), [ksyx](#), [lychees](#), [mcendu](#), [Molmin](#), [oosquare](#), [ouuan](#), [shuzhouliu](#), [sshwy](#), [StudyingFather](#), [thredreams](#), [Tiphereth-A](#), [Xeonacid](#), [zhu-yifang](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用

